

# Midterms 2 Revision

## Queues, Linked Lists and Trees

Step-wise algorithms · C examples · Worked practice

### What this revision guide covers

The supplied syllabus photograph lists **queues, linked lists, trees, binary trees and binary search trees**. Your request also explicitly includes **singly and doubly linked lists**. These are the core topics here. Circular queues receive full treatment because they fix a central limitation of array queues. Circular lists, deques and priority queues receive short recognition notes. Advanced balanced trees, graphs, hashing and sorting are outside this guide.

The photograph provides topic names, not lecture notes or an exam marking scheme. This guide therefore builds the standard foundations and common exam operations from scratch. It supports thorough preparation; exact coverage and marks still depend on your course's questions and conventions.

### How to use it

1. Read each definition beside its diagram. Say what each arrow and variable means.
2. Trace every algorithm on an empty structure, a one-item structure and a normal example.
3. Cover the worked answers and reproduce the result yourself.
4. Attempt the final practice questions before opening the answer section.
5. Use the last-page checklist immediately before the exam.

### Conventions used throughout

- Array indices start at **0**; linked-list positions start at **1**.
- NULL means a pointer does not refer to a node. It is not a node containing zero.
- Tree depth and height count **edges**: root depth 0, leaf height 0, empty-tree height  $-1$ . Some courses count nodes instead; state your convention.
- BST keys are distinct. Repeated keys are rejected unless a question says otherwise.
- In algorithms,  $\leftarrow$  means assignment. "Visit" means process or print the current item.
- Every numbered algorithm begins with **Start** and finishes with **End**. An early exit explicitly jumps to the last step.

### What a complete exam answer usually needs

Give the definition, state the representation and starting values, write clear numbered steps, handle special cases, draw a short example, and state time and extra-space costs. A correct final picture alone does not explain the pointer changes that earned it.

## Contents

|    |                                                     |    |
|----|-----------------------------------------------------|----|
| 1  | Foundations: data, addresses and running time       | 4  |
| 2  | Queues: FIFO and a simple array implementation      | 5  |
| 3  | Circular queues: reuse the freed space              | 6  |
| 4  | Circular queue: a complete trace and C example      | 7  |
| 5  | Linked queues and other queue forms                 | 9  |
| 6  | Singly linked lists: understand the links first     | 11 |
| 7  | Singly linked lists: insertion                      | 12 |
| 8  | Singly linked lists: deletion                       | 14 |
| 9  | Singly linked lists: position deletion and reversal | 16 |
| 10 | C patterns for safe singly linked-list operations   | 18 |
| 11 | Doubly linked lists: links in both directions       | 19 |
| 12 | Doubly linked lists: middle insertion and deletion  | 20 |
| 13 | List variants, reversal and comparisons             | 22 |
| 14 | Trees: build the vocabulary from one picture        | 23 |
| 15 | Binary trees: shapes and useful counting rules      | 24 |
| 16 | How binary trees are stored                         | 25 |
| 17 | Tree traversals: visit every node in a chosen order | 26 |
| 18 | Traversals: worked example and the queue connection | 27 |
| 19 | Tree measurements and reconstruction practice       | 29 |
| 20 | Binary search trees: the ordering rule and search   | 30 |
| 21 | BST insertion, minimum, maximum and successor       | 31 |
| 22 | BST deletion: understand the three cases            | 32 |
| 23 | BST deletion: illustrated worked examples           | 33 |
| 24 | C example: BST search, deletion and cleanup         | 34 |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>25 Fast revision: costs and common traps</b>              | <b>35</b> |
| <b>26 Practice paper: queues and linked lists (30 marks)</b> | <b>36</b> |
| <b>27 Practice paper: trees and BSTs (30 marks)</b>          | <b>37</b> |
| <b>28 Detailed answers: Q1 and Q2</b>                        | <b>38</b> |
| <b>29 Detailed answers: Q3 and Q4</b>                        | <b>39</b> |
| <b>30 Detailed answers: Q5 and Q6</b>                        | <b>40</b> |
| <b>31 Detailed answers: Q7 and Q8</b>                        | <b>41</b> |
| <b>32 Detailed answers: Q9, three successive deletions</b>   | <b>42</b> |
| <b>33 Detailed answers: Q10 and final self-check</b>         | <b>43</b> |

# 1 Foundations: data, addresses and running time

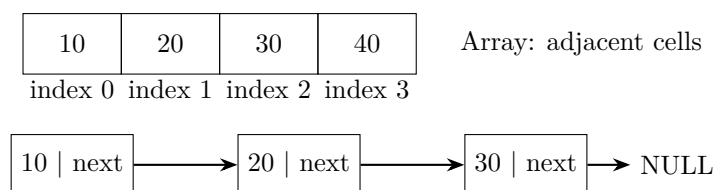
## Data structures in plain language

A **data structure** is a way to arrange information so that we can use it efficiently. A queue arranges items by arrival order; a linked list connects items through addresses; a tree arranges items in a hierarchy.

An **operation** is an action such as inserting, deleting, searching or visiting items. An **algorithm** is a precise sequence of steps for performing an operation. A structure's **invariant** is a rule that must remain true after every operation, such as "the last node's next pointer is NULL."

## Arrays versus linked nodes

An array stores elements consecutively. Indexing `a[i]` goes directly to one position. A linked structure stores separate nodes; each node contains data and one or more pointers to other nodes. Consecutive logical nodes need not occupy consecutive memory addresses.



## Only the C pointer ideas you need

- Node `*p`; declares a pointer to a node; initialise it before use.
- `p->data` accesses the data inside the node pointed to by `p`.
- `p = p->next`; moves the pointer to the next node; it does not move nodes.
- `malloc(sizeof *p)` requests space for one node. Check for NULL before using it.
- `free(p)` releases a dynamically allocated node. Do not access that node afterwards.
- Save needed links before overwriting them or freeing a node. Otherwise the rest of the structure may become unreachable.

## Time and space: what the symbols mean

Let  $n$  be the number of stored items,  $h$  a tree's height, and  $C$  an array's capacity.

| Cost        | Meaning                                             | Typical example             |
|-------------|-----------------------------------------------------|-----------------------------|
| $O(1)$      | Bounded work independent of $n$                     | Insert at a known list head |
| $O(\log n)$ | Work grows slowly as the problem repeatedly shrinks | Search a balanced BST       |
| $O(n)$      | Work may grow in proportion to all items            | Search an unsorted list     |
| $O(h)$      | Work follows at most one tree path                  | BST search                  |

Unless stated otherwise, costs are worst-case. **Auxiliary space** means temporary memory used by an operation, excluding the existing structure. A recursive call uses stack space too.  $O$  describes an upper bound; a tighter analysis may use  $\Theta$ .

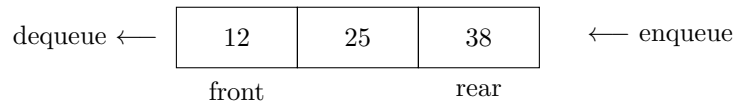
**Worked checkpoint.** Does deleting a node always take  $O(1)$ ?

**Answer.** No. The pointer changes may take  $O(1)$ , but locating the node or its predecessor can take  $O(n)$ . State what is already known before claiming a constant-time operation.

## 2 Queues: FIFO and a simple array implementation

### The idea

A **queue** follows **FIFO**: first in, first out. Think of people waiting at a counter. **Enqueue** adds at the **rear**; **dequeue** removes from the **front**. **Peek** reads the front item without removing it. Applications include print jobs, task scheduling and level-order tree traversal.



**Underflow**: attempting to remove from an empty queue. **Overflow**: attempting to insert when the chosen representation cannot accept another item. A linked queue can also fail if memory allocation fails.

### Linear array queue: one consistent convention

Use  $Q[0..C-1]$ , with  $\text{front} = \text{rear} = -1$  initially and  $C > 0$ . Both pointers are indices of occupied cells. Empty means  $\text{front} == -1$ . Full for this implementation means  $\text{rear} == C-1$ . Reset both indices after removing the final item.

#### Algorithm: Enqueue $x$ in a linear array queue

- Step 1.** Start.
- Step 2.** If  $\text{rear} = C - 1$ , report overflow and go to Step 6.
- Step 3.** If  $\text{front} = -1$ , set  $\text{front} \leftarrow 0$ .
- Step 4.** Set  $\text{rear} \leftarrow \text{rear} + 1$ .
- Step 5.** Set  $Q[\text{rear}] \leftarrow x$ .
- Step 6.** End.

#### Algorithm: Dequeue from a linear array queue

- Step 1.** Start.
- Step 2.** If  $\text{front} = -1$ , report underflow and go to Step 6.
- Step 3.** Save  $x \leftarrow Q[\text{front}]$  as the removed value.
- Step 4.** If  $\text{front} = \text{rear}$ , set  $\text{front} \leftarrow -1$  and  $\text{rear} \leftarrow -1$ ; otherwise set  $\text{front} \leftarrow \text{front} + 1$ .
- Step 5.** Output  $x$  as a successful removal.
- Step 6.** End.

Both operations take  $O(1)$  time and  $O(1)$  auxiliary space. The array itself occupies  $O(C)$  space. Peek checks for empty, then reads  $Q[\text{front}]$ ; it changes no indices.

---

**Worked checkpoint.** Capacity is 5. Enqueue 10, 20, 30, 40, 50; dequeue twice. Can this linear implementation enqueue 60?

**Answer.** No. The live items are 30, 40, 50 at indices 2, 3, 4. Although indices 0 and 1 are unused, **rear** is still 4. This is often called **false overflow** or wasted space. Shifting items left costs  $O(n)$ ; a circular queue reuses the space without shifting.

### 3 Circular queues: reuse the freed space

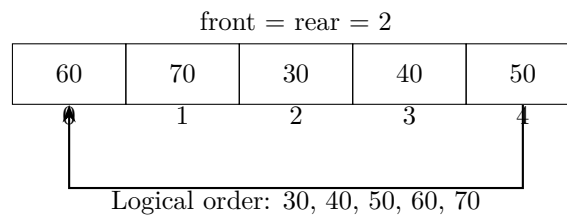
#### Wrap around, but preserve FIFO

A **circular queue** treats the cell after index  $C - 1$  as index 0. The calculation

$$\text{next}(i) = (i + 1) \bmod C$$

does the wrapping. This changes where items sit in memory, not their arrival order.

We use **front**, **rear** and **count**: **front** is the next removal index, **rear** is the next insertion index, and **count** is the number of live items. Initially all three are 0. Empty means **count** == 0; full means **count** ==  $C$ . All  $C$  cells are usable.



Here **front** = **rear** = 2 and **count** = 5: the queue is full. Rear identifies the next insertion index, but insertion is blocked until a cell is freed. The next page traces how this state is reached.

#### Algorithm: Enqueue $x$ in a circular array queue

- Step 1.** Start.
- Step 2.** If  $\text{count} = C$ , report overflow and go to Step 6.
- Step 3.** Set  $Q[\text{rear}] \leftarrow x$ .
- Step 4.** Set  $\text{rear} \leftarrow (\text{rear} + 1) \bmod C$ .
- Step 5.** Set  $\text{count} \leftarrow \text{count} + 1$ .
- Step 6.** End.

#### Algorithm: Dequeue from a circular array queue

- Step 1.** Start.
- Step 2.** If  $\text{count} = 0$ , report underflow and go to Step 7.
- Step 3.** Save  $x \leftarrow Q[\text{front}]$ .
- Step 4.** Set  $\text{front} \leftarrow (\text{front} + 1) \bmod C$ .
- Step 5.** Set  $\text{count} \leftarrow \text{count} - 1$ .
- Step 6.** Output  $x$  as a successful removal.
- Step 7.** End.

**Why count matters:**  $\text{front} = \text{rear}$  can mean empty *or* full. Count distinguishes them. Another valid design reserves one unused cell and tests  $(\text{rear} + 1) \bmod C = \text{front}$  for full; that design stores only  $C - 1$  items. Do not combine its full condition with this guide's count-based design.

**Exam habit:** State whether rear points to the last item or to the next free cell. Both conventions exist; algorithms are marked for consistency.

## 4 Circular queue: a complete trace and C example

Trace with capacity  $C = 5$

| Operation                  | front | rear | count | FIFO order         |
|----------------------------|-------|------|-------|--------------------|
| Initial                    | 0     | 0    | 0     | empty              |
| Enqueue 10, 20, 30, 40, 50 | 0     | 0    | 5     | 10, 20, 30, 40, 50 |
| Dequeue: returns 10        | 1     | 0    | 4     | 20, 30, 40, 50     |
| Dequeue: returns 20        | 2     | 0    | 3     | 30, 40, 50         |
| Enqueue 60                 | 2     | 1    | 4     | 30, 40, 50, 60     |
| Enqueue 70                 | 2     | 2    | 5     | 30, 40, 50, 60, 70 |
| Enqueue 80: overflow       | 2     | 2    | 5     | unchanged          |

To display the live items, visit  $Q[(front + k) \bmod C]$  for  $k = 0, 1, \dots, count - 1$ . Old values outside this range are not queue contents and need not be erased.

### C implementation of the same convention

This is a self-contained function group, without a `main`. A return value of 1 means success; 0 means failure. An output pointer must point to a valid `int`. Separating status from data allows the queue to store values such as  $-1$  safely.

```
#define CAP 5
typedef struct {
    int data[CAP];
    int front, rear, count;
} Queue;

void initQueue(Queue *q) {
    q->front = q->rear = q->count = 0;
}

int enqueue(Queue *q, int value) {
    if (q->count == CAP) return 0;
    q->data[q->rear] = value;
    q->rear = (q->rear + 1) % CAP;
    q->count++;
    return 1;
}

int dequeue(Queue *q, int *out) {
    if (q->count == 0) return 0;
    *out = q->data[q->front];
    q->front = (q->front + 1) % CAP;
    q->count--;
    return 1;
}

int peekQueue(const Queue *q, int *out) {
    if (q->count == 0) return 0;
    *out = q->data[q->front];
    return 1;
}

/* Example inside main:
Queue q; int value;
initQueue(&q);
enqueue(&q, 42);
if (dequeue(&q, &value)) printf("%d\n", value);
Include <stdio.h> if using printf.
*/
```

---

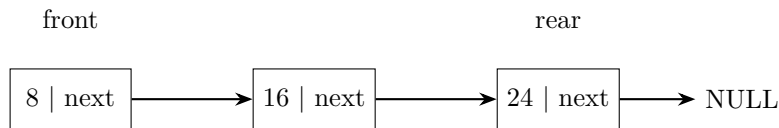
**Worked checkpoint.** After all five items are removed from the final full state above, what are the indices?

**Answer.** Both indices are 2 and count is 0. Empty does not require the indices to become zero. Five removals advance front once around the array.

## 5 Linked queues and other queue forms

### A queue made from linked nodes

Keep two pointers: **front** to the first node and **rear** to the last. Initially both are NULL. Every node stores data and **next**; the last node's **next** is NULL. This avoids a fixed array capacity, although memory is still limited.



#### Algorithm: Enqueue $x$ in a linked queue

- Step 1.** Start.
- Step 2.** Allocate node  $N$ . If allocation fails, report failure and go to Step 7.
- Step 3.** Set  $N.data \leftarrow x$  and  $N.next \leftarrow \text{NULL}$ .
- Step 4.** If  $front = \text{NULL}$ , set  $front \leftarrow N$  and  $rear \leftarrow N$ , then go to Step 7.
- Step 5.** Set  $rear.next \leftarrow N$ .
- Step 6.** Set  $rear \leftarrow N$ .
- Step 7.** End.

#### Algorithm: Dequeue from a linked queue

- Step 1.** Start.
- Step 2.** If  $front = \text{NULL}$ , report underflow and go to Step 8.
- Step 3.** Set  $T \leftarrow front$  and save  $x \leftarrow T.data$ .
- Step 4.** Set  $front \leftarrow front.next$ .
- Step 5.** If  $front = \text{NULL}$ , set  $rear \leftarrow \text{NULL}$  too.
- Step 6.** Free  $T$ .
- Step 7.** Output  $x$  as a successful removal.
- Step 8.** End.

Both operations take  $O(1)$  time under the usual allocation model. Total storage is  $O(n)$ . Keeping **rear** avoids walking through the entire list for every insertion.

### Recognise these variants

| Form                    | Distinguishing rule                                                  |
|-------------------------|----------------------------------------------------------------------|
| Linear queue            | FIFO; insert at rear, remove at front.                               |
| Circular queue          | FIFO with array indices that wrap around.                            |
| Deque                   | Double-ended queue: insert and remove at both ends.                  |
| Input-restricted deque  | Insert at one end; remove at either end.                             |
| Output-restricted deque | Insert at either end; remove at one end.                             |
| Priority queue          | Remove by priority, not necessarily arrival time; define a tie rule. |

---

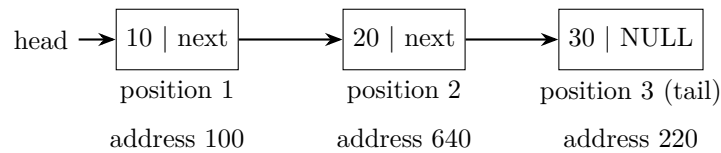
**Worked checkpoint.** A linked queue contains one node. Why must dequeue set rear to NULL?

**Answer.** Once the node is freed, front becomes NULL. If rear still points to the freed node, it is a dangling pointer. The invariant “empty means both pointers are NULL” prevents later insertion from using invalid memory.

## 6 Singly linked lists: understand the links first

### Node, head, link and tail

A **singly linked list** is a sequence of nodes. Each node stores **data** and one **next pointer**. The **head** points to the first node. The **tail** is the last node, whose next pointer is **NULL**. An empty list has **head == NULL**. A separate tail pointer is optional; the algorithms in this section maintain only head.



The addresses are illustrative. The first node's next field stores the second node's address, 640, not its data value 20. Pointer links, rather than physical placement, define the order.

```

#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;
/* Empty list: Node *head = NULL; */

```

The tag **struct Node** allows the structure to contain a pointer to its own type. The node does not contain another complete node inside it.

### Algorithm: Traverse or display a singly linked list

**Step 1.** Start.

**Step 2.** Set  $P \leftarrow head$ .

**Step 3.** While  $P \neq \text{NULL}$ , visit  $P.data$  and then set  $P \leftarrow P.next$ .

**Step 4.** End.

### Algorithm: Search for the first occurrence of key $k$

**Step 1.** Start.

**Step 2.** Set  $P \leftarrow head$  and  $position \leftarrow 1$ .

**Step 3.** While  $P \neq \text{NULL}$ , do the following: if  $P.data = k$ , report  $position$  and go to Step 5; otherwise set  $P \leftarrow P.next$  and  $position \leftarrow position + 1$ .

**Step 4.** Report “not found.”

**Step 5.** End.

Traversal takes  $O(n)$  time. Search takes  $O(1)$  best-case time if the head matches, and  $O(n)$  worst-case time. Both use  $O(1)$  auxiliary space.

**Worked checkpoint.** In the picture, after  $P = head$ ;  $P = P \rightarrow next$ ;, what is  $P \rightarrow data$ ? Does head change?

**Answer.** The value is 20. Head still points to 10 because only the local pointer  $P$  moved. If head itself were moved repeatedly, the original first-node address could be lost.

## 7 Singly linked lists: insertion

### Insert at the beginning

Connect the new node to the old head *before* changing head.

#### Algorithm: Insert $x$ at the beginning

**Step 1.** Start.

**Step 2.** Allocate node  $N$ . If allocation fails, report failure and go to Step 6.

**Step 3.** Set  $N.data \leftarrow x$ .

**Step 4.** Set  $N.next \leftarrow head$ .

**Step 5.** Set  $head \leftarrow N$ .

**Step 6.** End.

This also handles an empty list because the old head is NULL. Time is  $O(1)$ .

#### Algorithm: Insert $x$ at the end, with only a head pointer

**Step 1.** Start.

**Step 2.** Allocate  $N$ . If allocation fails, report failure and go to Step 7.

**Step 3.** Set  $N.data \leftarrow x$  and  $N.next \leftarrow \text{NULL}$ .

**Step 4.** If  $head = \text{NULL}$ , set  $head \leftarrow N$  and go to Step 7.

**Step 5.** Set  $P \leftarrow head$ . While  $P.next \neq \text{NULL}$ , set  $P \leftarrow P.next$ .

**Step 6.** Set  $P.next \leftarrow N$ .

**Step 7.** End.

Time is  $O(n)$  with only head; it becomes  $O(1)$  if a valid tail pointer is also maintained.

#### Algorithm: Insert $x$ at position $p$ (valid range 1 to $n + 1$ )

**Step 1.** Start.

**Step 2.** If  $p < 1$ , report an invalid position and go to Step 8.

**Step 3.** If  $p = 1$ , perform the beginning-insertion algorithm and go to Step 8.

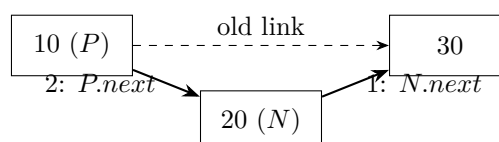
**Step 4.** Set  $P \leftarrow head$ ,  $i \leftarrow 1$ . While  $P \neq \text{NULL}$  and  $i < p - 1$ , set  $P \leftarrow P.next$  and  $i \leftarrow i + 1$ .

**Step 5.** If  $P = \text{NULL}$ , report an invalid position and go to Step 8.

**Step 6.** Allocate  $N$ . If allocation fails, report failure and go to Step 8. Set  $N.data \leftarrow x$ .

**Step 7.** Set  $N.next \leftarrow P.next$ , then set  $P.next \leftarrow N$ .

**Step 8.** End.



---

**Worked checkpoint.** Insert 20 at position 2 in  $10 \rightarrow 30 \rightarrow \text{NULL}$ . Why is link order important?

**Answer.** Save the old successor through  $N \rightarrow \text{next} = P \rightarrow \text{next}$ , then connect P to N. Reversing those two assignments makes  $N \rightarrow \text{next}$  point to N itself and loses the original link to 30.

## 8 Singly linked lists: deletion

Deletion must detach a node, preserve the remaining links, and free the removed node exactly once. The following algorithms update head when required.

### Algorithm: Delete the first node

- Step 1.** Start.
- Step 2.** If  $head = \text{NULL}$ , report an empty list and go to Step 6.
- Step 3.** Set  $T \leftarrow head$  and save  $x \leftarrow T.data$ .
- Step 4.** Set  $head \leftarrow T.next$ .
- Step 5.** Free  $T$  and output  $x$ .
- Step 6.** End.

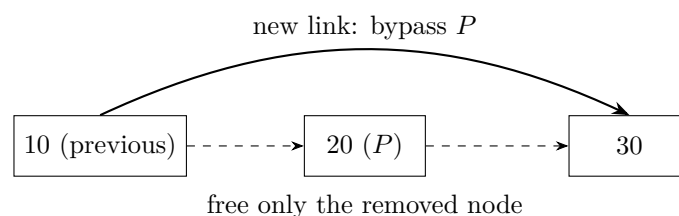
### Algorithm: Delete the last node, with only a head pointer

- Step 1.** Start.
- Step 2.** If  $head = \text{NULL}$ , report an empty list and go to Step 7.
- Step 3.** If  $head.next = \text{NULL}$ , save its data, free head, set  $head \leftarrow \text{NULL}$ , output the saved data and go to Step 7.
- Step 4.** Set  $P \leftarrow head$ . While  $P.next.next \neq \text{NULL}$ , set  $P \leftarrow P.next$ .
- Step 5.** Set  $T \leftarrow P.next$ , save  $x \leftarrow T.data$ , and set  $P.next \leftarrow \text{NULL}$ .
- Step 6.** Free  $T$  and output  $x$ .
- Step 7.** End.

Step 4 is safe because the earlier checks guarantee at least two nodes. The loop stops at the node just before the tail. Time:  $O(n)$ .

### Algorithm: Delete the first node whose data equals $k$

- Step 1.** Start.
- Step 2.** Set  $P \leftarrow head$  and  $previous \leftarrow \text{NULL}$ .
- Step 3.** While  $P \neq \text{NULL}$  and  $P.data \neq k$ , set  $previous \leftarrow P$  and then  $P \leftarrow P.next$ .
- Step 4.** If  $P = \text{NULL}$ , report “not found” and go to Step 8.
- Step 5.** If  $previous = \text{NULL}$ , set  $head \leftarrow P.next$ ; otherwise set  $previous.next \leftarrow P.next$ .
- Step 6.** Save  $x \leftarrow P.data$  and free  $P$ .
- Step 7.** Output  $x$  as a successful removal.
- Step 8.** End.



---

**Worked checkpoint.** Delete 7 from  $7 \rightarrow 7 \rightarrow 9 \rightarrow \text{NULL}$  using this algorithm.

**Answer.** The result is  $7 \rightarrow 9 \rightarrow \text{NULL}$ . Only the first matching node is deleted. Its predecessor is NULL, so head must change. To delete every match, a different repeated-deletion procedure is needed.

**Exam habit:** A tail pointer alone does not make singly linked tail deletion  $O(1)$ : you still need the predecessor.

## 9 Singly linked lists: position deletion and reversal

**Algorithm: Delete the node at position  $p$  (valid range 1 to  $n$ )**

**Step 1.** Start.

**Step 2.** If  $p < 1$  or  $head = \text{NULL}$ , report failure and go to Step 8.

**Step 3.** If  $p = 1$ , perform the first-node deletion algorithm and go to Step 8.

**Step 4.** Set  $P \leftarrow head$  and  $i \leftarrow 1$ . While  $P \neq \text{NULL}$  and  $i < p - 1$ , set  $P \leftarrow P.next$  and  $i \leftarrow i + 1$ .

**Step 5.** If  $P = \text{NULL}$  or  $P.next = \text{NULL}$ , report an invalid position and go to Step 8.

**Step 6.** Set  $T \leftarrow P.next$ , save  $x \leftarrow T.data$ , and set  $P.next \leftarrow T.next$ .

**Step 7.** Free  $T$  and output  $x$ .

**Step 8.** End.

### Reverse links, rather than swapping the data

Keep three pointers: previous is the reversed part, current is the next node to reverse, and savedNext remembers the rest of the original list.

**Algorithm: Reverse a singly linked list in place**

**Step 1.** Start.

**Step 2.** Set  $previous \leftarrow \text{NULL}$  and  $current \leftarrow head$ .

**Step 3.** While  $current \neq \text{NULL}$ , perform these assignments *in order*:  $savedNext \leftarrow current.next$ ;  $current.next \leftarrow previous$ ;  $previous \leftarrow current$ ;  $current \leftarrow savedNext$ .

**Step 4.** Set  $head \leftarrow previous$ .

**Step 5.** End.

| Stage              | Reversed part (previous) | Unprocessed part (current) |
|--------------------|--------------------------|----------------------------|
| Initially          | NULL                     | 10 → 20 → 30 → NULL        |
| After 1 iteration  | 10 → NULL                | 20 → 30 → NULL             |
| After 2 iterations | 20 → 10 → NULL           | 30 → NULL                  |
| After 3 iterations | 30 → 20 → 10 → NULL      | NULL                       |

Each iteration grows the reversed part by one node while preserving the remaining part. Time is  $O(n)$ ; auxiliary space is  $O(1)$ . Empty and one-node lists work without special cases. If a separate tail pointer exists, the original head becomes the new tail.

```

/* Uses the Node definition from the singly linked-list section. */
Node *reverseList(Node *head) {
    Node *previous = NULL;
    Node *current = head;
    while (current != NULL) {
        Node *savedNext = current->next;
        current->next = previous;
        previous = current;
        current = savedNext;
    }
}

```

```
    }  
    return previous;  
}  
/* Caller must retain the new head: head = reverseList(head); */
```

---

**Worked checkpoint.** Why save current's next pointer first?

**Answer.** Reversing the link overwrites the only forward route to the unprocessed nodes. savedNext preserves that route before the overwrite.

## 10 C patterns for safe singly linked-list operations

The functions below use the earlier `Node` definition and `<stdlib.h>`. Passing `Node **head` lets a function change the caller's head pointer. Call with `&head`; `*head` means the actual first-node pointer. The argument itself must be valid.

```
int pushFront(Node **head, int value) {
    Node *node = malloc(sizeof *node);
    if (node == NULL) return 0;
    node->data = value;
    node->next = *head;
    *head = node;
    return 1;
}

int removeFirst(Node **head, int key) {
    Node *current = *head;
    Node *previous = NULL;
    while (current != NULL && current->data != key) {
        previous = current;
        current = current->next;
    }
    if (current == NULL) return 0;
    if (previous == NULL)
        *head = current->next;
    else
        previous->next = current->next;
    free(current);
    return 1;
}

void clearList(Node **head) {
    Node *current = *head;
    while (current != NULL) {
        Node *savedNext = current->next;
        free(current);
        current = savedNext;
    }
    *head = NULL;
}

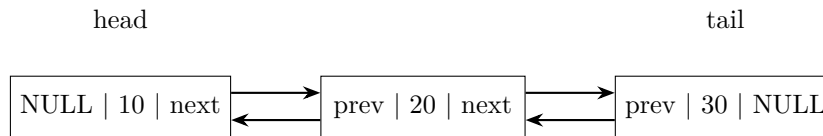
/* Example inside main:
Node *head = NULL;
if (!pushFront(&head, 10)) { handle allocation failure; }
removeFirst(&head, 10);
clearList(&head);
*/
```

### Find the mistake before it costs marks

| Mistake                                                                                      | Why it fails / correction                                                                                                               |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>free(p); p = p-&gt;next;</code><br><code>while (p-&gt;next != NULL) for display</code> | Reads a freed node. Save next before calling free.<br>Skips the tail and is unsafe when p is NULL. Use <code>while (p != NULL)</code> . |
| Changing only a local copy of head                                                           | The caller keeps the old head. Return the new head or pass its address.                                                                 |
| Failing to free an unlinked node                                                             | The storage remains allocated: a memory leak.                                                                                           |
| Using <code>p-&gt;data</code> without checking p                                             | A NULL pointer cannot be dereferenced.                                                                                                  |

## 11 Doubly linked lists: links in both directions

A **doubly linked list** stores **prev**, **data**, and **next** in every node. It supports forward and backward movement. Here we maintain **both head and tail**.



**Rules to preserve:**  $\text{head.prev}$  is NULL,  $\text{tail.next}$  is NULL, and if  $A.\text{next} = B$  then  $B.\text{prev} = A$ . In an empty list both head and tail are NULL. In a one-node list both refer to that node, whose two links are NULL.

```

typedef struct DNode {
    int data;
    struct DNode *prev, *next;
} DNode;

```

**Algorithm: Insert  $x$  at the beginning of a doubly linked list**

- Step 1.** Start.
- Step 2.** Allocate  $N$ . If allocation fails, report failure and go to Step 7.
- Step 3.** Set  $N.\text{data} \leftarrow x$ ,  $N.\text{prev} \leftarrow \text{NULL}$  and  $N.\text{next} \leftarrow \text{head}$ .
- Step 4.** If  $\text{head} \neq \text{NULL}$ , set  $\text{head}.\text{prev} \leftarrow N$ .
- Step 5.** Otherwise set  $\text{tail} \leftarrow N$  (the list was empty).
- Step 6.** Set  $\text{head} \leftarrow N$ .
- Step 7.** End.

**Algorithm: Insert  $x$  at the end of a doubly linked list**

- Step 1.** Start.
- Step 2.** Allocate  $N$ . If allocation fails, report failure and go to Step 7.
- Step 3.** Set  $N.\text{data} \leftarrow x$ ,  $N.\text{next} \leftarrow \text{NULL}$  and  $N.\text{prev} \leftarrow \text{tail}$ .
- Step 4.** If  $\text{tail} \neq \text{NULL}$ , set  $\text{tail}.\text{next} \leftarrow N$ .
- Step 5.** Otherwise set  $\text{head} \leftarrow N$  (the list was empty).
- Step 6.** Set  $\text{tail} \leftarrow N$ .
- Step 7.** End.

Both take  $O(1)$  time because the required end pointer is already available.

**Worked checkpoint.** A new node 5 is inserted before the only node 10. Which links change?

**Answer.** Set 5.prev to NULL and 5.next to 10; set 10.prev to 5; set head to 5. Tail stays at 10. Forgetting 10.prev leaves forward traversal correct but backward traversal broken.

## 12 Doubly linked lists: middle insertion and deletion

**Algorithm: Insert  $x$  immediately after a known node  $P$**

**Step 1.** Start.

**Step 2.** If  $P = \text{NULL}$ , report an invalid node and go to Step 8. Otherwise assume  $P$  belongs to this list.

**Step 3.** Allocate  $N$ . If allocation fails, report failure and go to Step 8.

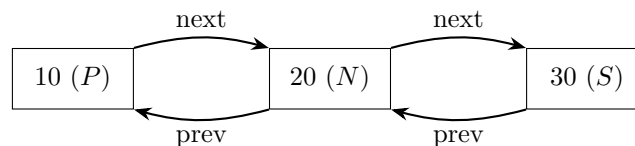
**Step 4.** Set  $N.data \leftarrow x$ ,  $N.prev \leftarrow P$  and  $N.next \leftarrow P.next$ .

**Step 5.** If  $P.next \neq \text{NULL}$ , set  $P.next.prev \leftarrow N$ .

**Step 6.** Otherwise set  $tail \leftarrow N$ .

**Step 7.** Set  $P.next \leftarrow N$ .

**Step 8.** End.



**Algorithm: Delete a known node  $P$  from a doubly linked list**

**Step 1.** Start.

**Step 2.** If  $P = \text{NULL}$ , report failure and go to Step 7. Otherwise assume  $P$  belongs to this list.

**Step 3.** If  $P.prev \neq \text{NULL}$ , set  $P.prev.next \leftarrow P.next$ ; otherwise set  $head \leftarrow P.next$ .

**Step 4.** If  $P.next \neq \text{NULL}$ , set  $P.next.prev \leftarrow P.prev$ ; otherwise set  $tail \leftarrow P.prev$ .

**Step 5.** Save  $x \leftarrow P.data$ .

**Step 6.** Free  $P$  and output  $x$ .

**Step 7.** End.

This one algorithm covers first-node, last-node, middle-node and only-node deletion. For deletion by value or position, locate  $P$  first using traversal. Updating links costs  $O(1)$ , but finding  $P$  can cost  $O(n)$ .

**Trace: delete 20 from  $10 \leftrightarrow 20 \leftrightarrow 30$**

Set  $10.next$  to 30 and  $30.prev$  to 10. Then free 20. Both directions must bypass 20. Head remains 10 and tail remains 30.

```

/* DNode definition and <stdlib.h> are required.
   head and tail are valid addresses; p belongs to this list. */
void eraseDNode(DNode **head, DNode **tail, DNode *p) {
    if (p == NULL) return;
    if (p->prev != NULL) p->prev->next = p->next;
    else *head = p->next;
    if (p->next != NULL) p->next->prev = p->prev;
    else *tail = p->prev;
    free(p);
}

```

---

**Worked checkpoint.** What happens if P is the only node?

**Answer.** Its prev and next are both NULL. Step 3 makes head NULL; Step 4 makes tail NULL. The node is then freed, producing a valid empty list.

## 13 List variants, reversal and comparisons

### Algorithm: Reverse a doubly linked list with head and tail

**Step 1.** Start.

**Step 2.** Set  $P \leftarrow \text{head}$ .

**Step 3.** While  $P \neq \text{NULL}$ , save  $S \leftarrow P.\text{next}$ , set  $P.\text{next} \leftarrow P.\text{prev}$ , set  $P.\text{prev} \leftarrow S$ , and then set  $P \leftarrow S$ .

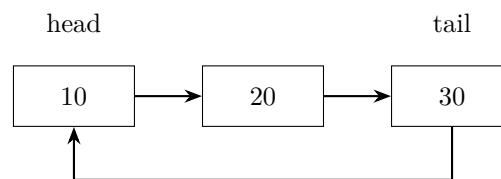
**Step 4.** Swap head and tail using a temporary pointer.

**Step 5.** End.

Every node's two directions are exchanged. Saving the original next link ensures the loop still follows the original list. Time is  $O(n)$  and auxiliary space is  $O(1)$ .

### Circular lists: recognise the stopping rule

In a **circular singly linked list**,  $\text{tail.next}$  points to head instead of NULL. In a circular doubly linked list,  $\text{tail.next}$  is head and  $\text{head.prev}$  is tail. They are useful for repeated cyclic turns, such as round-robin scheduling.



For traversal, first check for an empty list. Otherwise visit head, follow next, and stop when the pointer returns to head. Waiting for NULL would loop forever. In a one-node circular singly list, the node points to itself.

### What does each representation buy you?

| Feature               | Array                    | Singly list                                   | Doubly list      |
|-----------------------|--------------------------|-----------------------------------------------|------------------|
| Access item by index  | $O(1)$                   | $O(n)$                                        | $O(n)$           |
| Unsorted search       | $O(n)$                   | $O(n)$                                        | $O(n)$           |
| Insert at beginning   | $O(n)$ shifting          | $O(1)$                                        | $O(1)$           |
| Append                | $O(1)$ if spare capacity | $O(1)$ with tail; else $O(n)$                 | $O(1)$ with tail |
| Delete last           | $O(1)$                   | $O(n)$                                        | $O(1)$ with tail |
| Delete known node     | May require shifting     | Needs predecessor for general $O(1)$ deletion | $O(1)$           |
| Per-item link storage | None                     | One pointer                                   | Two pointers     |
| Backward movement     | By index                 | No direct link                                | Via prev         |

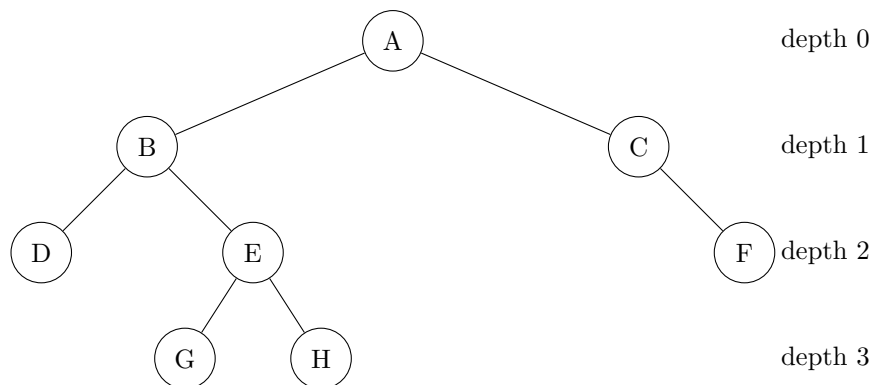
Array costs refer to an unsorted sequence with its length known. Dynamic-array resizing can make an individual append  $O(n)$ . Linked structures need extra pointer storage and typically have less predictable memory access than arrays.

**Worked checkpoint.** Choose a structure: frequent movement both forward and backward, and deletion of an already selected item.

**Answer.** A doubly linked list fits: prev and next allow both movements and local deletion in  $O(1)$ . The trade-off is the extra pointer in every node and more links to update correctly.

## 14 Trees: build the vocabulary from one picture

A **tree** represents a hierarchy. It consists of nodes connected by edges. In a rooted tree, one node is chosen as the root; every other node has exactly one parent. There is one path between any two nodes, and no cycle (no route that loops back without retracing an edge).



| Term                 | Meaning                                   | Example above          |
|----------------------|-------------------------------------------|------------------------|
| Node                 | One stored item or point in the structure | E                      |
| Edge                 | One direct connection between nodes       | B–E                    |
| Root                 | Top node with no parent                   | A                      |
| Parent               | Node directly above another node          | B is E's parent        |
| Child                | Node directly below a parent              | G is E's child         |
| Siblings             | Nodes with the same parent                | D and E                |
| Leaf / external node | Node with no children                     | D, G, H, F             |
| Internal node        | Node with at least one child              | A, B, C, E             |
| Ancestor             | Parent, parent's parent, and so on        | A and B of E           |
| Descendant           | Child, child's child, and so on           | E, G, H of B           |
| Subtree              | A node and all its descendants            | E with G and H         |
| Degree of a node     | Number of its children                    | $\text{degree}(E) = 2$ |
| Degree of a tree     | Largest node degree                       | 2                      |
| Path                 | Consecutive connected nodes               | A, B, E, G             |
| Path length          | Number of edges on that path              | 3 for A to G           |
| Depth of a node      | Edges from root to the node               | $\text{depth}(E) = 2$  |
| Height of a node     | Longest downward path to a leaf           | $\text{height}(B) = 2$ |
| Height of the tree   | Height of its root                        | 3                      |

We use **level** = **depth**, so the root is at level 0. A course using root level 1 will give every level a value one larger. Ancestors and descendants here exclude the node itself. “Degree” in this guide counts children, not all incident edges as in an undirected graph.

**Worked checkpoint.** How many nodes and edges are there? What is the subtree rooted at C?

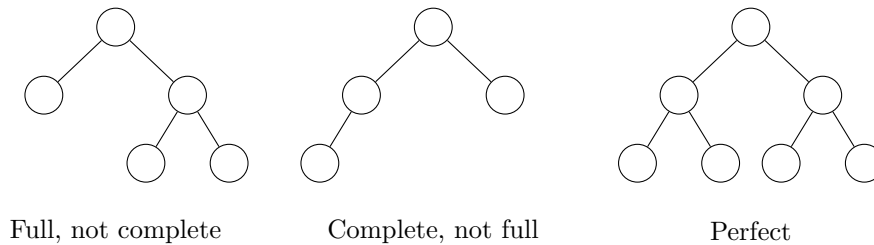
**Answer.** There are 8 nodes and 7 edges. Every non-root node contributes exactly one parent edge, so a nonempty tree with  $n$  nodes has  $n - 1$  edges. C's subtree contains C and F. C has degree 1, depth 1 and height 1.

## 15 Binary trees: shapes and useful counting rules

A **binary tree** is a tree in which each node has at most two children, distinguished as **left** and **right**. Having only a left child differs from having only a right child. A binary tree does *not* automatically order its stored values.

### Do not confuse these words

- **Full / strict / proper:** every node has either zero or two children.
- **Complete:** every level except possibly the last is full, and the last is filled from left to right without gaps.
- **Perfect:** every internal node has two children and all leaves are at the same depth. Every level is full.
- **Skewed:** each internal node has only one child, producing a chain; it may be left-skewed or right-skewed.
- **Height-balanced:** under the common local definition, left and right subtree heights differ by at most one *at every node*. Balancing algorithms are outside this syllabus scope.



### Formulas with height counted in edges

| Fact                               | Formula and reason                                               |
|------------------------------------|------------------------------------------------------------------|
| Maximum nodes at depth $d$         | $2^d$ : each node can have at most two children.                 |
| Maximum nodes at height $h \geq 0$ | $1 + 2 + \dots + 2^h = 2^{h+1} - 1$ . A perfect tree attains it. |
| Minimum nodes at height $h \geq 0$ | $h + 1$ , attained by a chain.                                   |
| Height range for $n \geq 1$ nodes  | $\lceil \log_2(n + 1) \rceil - 1 \leq h \leq n - 1$ .            |
| Perfect tree leaves                | $L = 2^h$ ; internal nodes $I = 2^h - 1$ .                       |
| Full binary tree                   | $L = I + 1$ and $n = 2I + 1 = 2L - 1$ .                          |
| Any nonempty binary tree           | $n_0 = n_2 + 1$ , where $n_j$ counts nodes with $j$ children.    |
| NULL child pointers                | $n + 1$ for $n$ real nodes: $2n$ slots minus $n - 1$ edges.      |

For the general identity, count edges in two ways:

$$n - 1 = n_1 + 2n_2, \quad n = n_0 + n_1 + n_2 \quad \implies \quad n_0 = n_2 + 1.$$

**Worked checkpoint.** A perfect binary tree has height 3. Find nodes and leaves. Can a binary tree of height 3 have only 4 nodes?

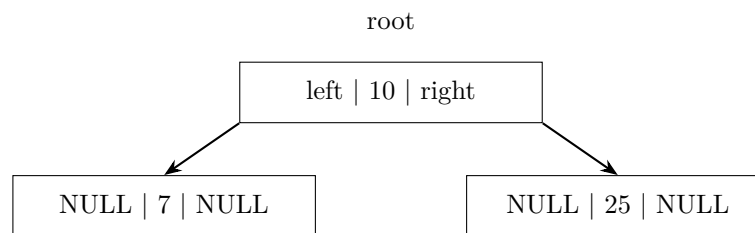
**Answer.** The perfect tree has  $2^4 - 1 = 15$  nodes and  $2^3 = 8$  leaves. Yes: a four-node chain also has height 3, but is neither perfect nor complete.

## 16 How binary trees are stored

### Linked representation

A node stores data, a left-child pointer and a right-child pointer. A missing child is NULL. Root points to the first node; an empty tree has root equal to NULL.

```
typedef struct TNode {
    int key;
    struct TNode *left, *right;
} TNode;
/* TNode *root = NULL; represents an empty tree. */
```

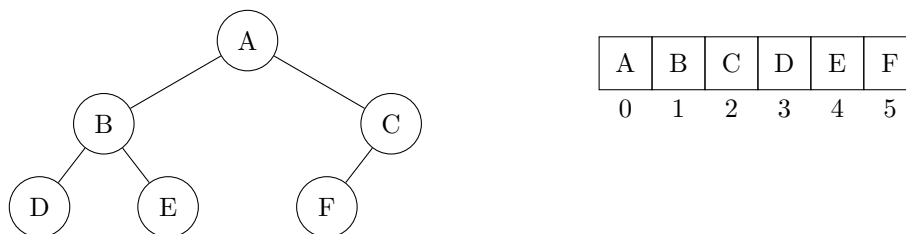


This representation is flexible for sparse or changing tree shapes. It needs two child pointers per node, and a normal node does not automatically know its parent.

### Array representation: especially suitable for complete trees

With **zero-based indexing**, store root at index 0. For a node at index  $i$ :

$$\text{left} = 2i + 1, \quad \text{right} = 2i + 2, \quad \text{parent} = \left\lfloor \frac{i - 1}{2} \right\rfloor \quad (i > 0).$$



For a complete tree with  $n$  nodes, a computed child exists when its index is less than  $n$ . In an arbitrary sparse tree, a valid index alone is insufficient; empty positions need an occupancy marker. Skewed trees waste many array positions under this mapping.

If your course uses **one-based indexing**, root is at 1, left is  $2i$ , right is  $2i + 1$ , and parent is  $\lfloor i/2 \rfloor$ . Write the convention before using a formula.

### Why general binary-tree search can be slow

The shape rule “at most two children” says nothing about key order. To find a value in an arbitrary binary tree, you may have to inspect every node:  $O(n)$  time. A BST adds an ordering rule that lets a search discard one subtree at each comparison.

**Worked checkpoint.** In the zero-based array above, find E’s parent and B’s children.

**Answer.** E is at index 4, so its parent index is  $\lfloor (4 - 1)/2 \rfloor = 1$ , which contains B. B is at 1; its children are at 3 and 4, containing D and E.

## 17 Tree traversals: visit every node in a chosen order

Traversal means visiting every node exactly once. Let N mean visit the current node, L mean traverse its left subtree, and R mean traverse its right subtree.

| Traversal   | Rule          | Plain-language order          | Common use                        |
|-------------|---------------|-------------------------------|-----------------------------------|
| Preorder    | NLR           | Node, left, right             | Describe/copy tree structure      |
| Inorder     | LNR           | Left, node, right             | Sorted keys of a BST              |
| Postorder   | LRN           | Left, right, node             | Free children before their parent |
| Level order | Breadth first | Level by level, left to right | Process nearest levels first      |

**Recursion** means applying the same procedure to a smaller subtree. The base case is an empty subtree: do nothing and return. “Return” resumes the caller; it does not terminate the entire traversal.

### Algorithm: Preorder( $T$ )

- Step 1.** Start.
- Step 2.** If  $T = \text{NULL}$ , go to Step 6.
- Step 3.** Visit  $T.data$ .
- Step 4.** Apply Preorder to  $T.left$ .
- Step 5.** Apply Preorder to  $T.right$ .
- Step 6.** End (return to the caller).

### Algorithm: Inorder( $T$ )

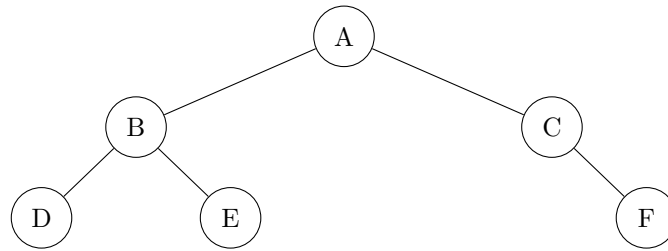
- Step 1.** Start.
- Step 2.** If  $T = \text{NULL}$ , go to Step 6.
- Step 3.** Apply Inorder to  $T.left$ .
- Step 4.** Visit  $T.data$ .
- Step 5.** Apply Inorder to  $T.right$ .
- Step 6.** End (return to the caller).

### Algorithm: Postorder( $T$ )

- Step 1.** Start.
- Step 2.** If  $T = \text{NULL}$ , go to Step 6.
- Step 3.** Apply Postorder to  $T.left$ .
- Step 4.** Apply Postorder to  $T.right$ .
- Step 5.** Visit  $T.data$ .
- Step 6.** End (return to the caller).

**Exam habit:** Apply the rule to whole subtrees, not just immediate children. Inorder is sorted only when the tree satisfies the BST ordering rule.

## 18 Traversals: worked example and the queue connection



- **Preorder:** A, B, D, E, C, F. Visit A first, then all of B's subtree, then all of C's subtree.
- **Inorder:** D, B, E, A, C, F. A appears after its entire left subtree; C appears before its right child F.
- **Postorder:** D, E, B, F, C, A. Each parent appears after all its descendants.
- **Level order:** A, B, C, D, E, F. Finish one depth before the next.

### Algorithm: Level-order traversal using a queue of node pointers

**Step 1.** Start.

**Step 2.** If  $root = \text{NULL}$ , go to Step 6.

**Step 3.** Create an empty queue and enqueue root.

**Step 4.** While the queue is not empty: dequeue a node pointer  $P$ ; visit  $P.data$ ; if  $P.left \neq \text{NULL}$ , enqueue it; if  $P.right \neq \text{NULL}$ , enqueue it.

**Step 5.** Release any queue storage used. The queue must have enough capacity, or grow as needed.

**Step 6.** End.

| Node removed | Children enqueued | Queue afterwards (front first) |
|--------------|-------------------|--------------------------------|
| Initially    | A                 | A                              |
| A            | B, C              | B, C                           |
| B            | D, E              | C, D, E                        |
| C            | F                 | D, E, F                        |
| D            | none              | E, F                           |
| E            | none              | F                              |
| F            | none              | empty                          |

### Costs and simple C

All four traversals take  $O(n)$  time. Recursive traversals use  $O(h + 1)$  stack space:  $O(\log n)$  on a balanced tree and  $O(n)$  on a chain. Level order uses  $O(w)$  queue space, where  $w$  is the maximum number of nodes on a level; in the worst case this is  $O(n)$ .

```

/* Requires <stdio.h> and the TNode definition. */
void inorder(const TNode *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->key);
    inorder(root->right);
}

```

---

**Worked checkpoint.** Why is postorder suitable for destroying a tree?

**Answer.** Free the left subtree, then the right subtree, then the current node. Freeing the parent first would make reading its child pointers unsafe. Printing and freeing are different visit actions, but the required order is the same.

## 19 Tree measurements and reconstruction practice

**Algorithm: Count all nodes in a binary tree:  $\text{Count}(T)$**

**Step 1.** Start.

**Step 2.** If  $T = \text{NULL}$ , set result to 0 and go to Step 5.

**Step 3.** Obtain  $L \leftarrow \text{Count}(T.\text{left})$  and  $R \leftarrow \text{Count}(T.\text{right})$ .

**Step 4.** Set result to  $1 + L + R$ .

**Step 5.** End and return result.

**Algorithm: Find the height in edges:  $\text{Height}(T)$**

**Step 1.** Start.

**Step 2.** If  $T = \text{NULL}$ , set result to  $-1$  and go to Step 5.

**Step 3.** Obtain  $L \leftarrow \text{Height}(T.\text{left})$  and  $R \leftarrow \text{Height}(T.\text{right})$ .

**Step 4.** Set result to  $1 + \max(L, R)$ .

**Step 5.** End and return result.

For a leaf, both subtrees have height  $-1$ , giving  $1 + \max(-1, -1) = 0$ . Both algorithms visit every node, take  $O(n)$  time, and use  $O(h + 1)$  recursive stack space.

### Counting leaves

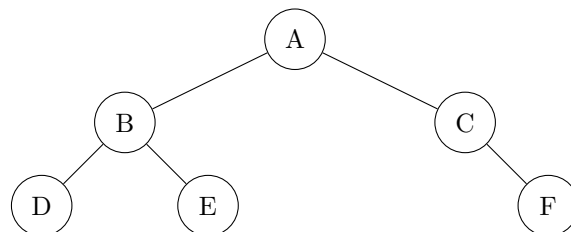
An empty subtree contributes 0 leaves. A node with *both* child pointers `NULL` contributes 1 leaf. Otherwise add the leaf counts of its two subtrees. Testing whether either child is `NULL` incorrectly counts one-child nodes as leaves.

### Reconstructing from preorder and inorder

For distinct node labels, preorder plus inorder determines a unique binary tree. The first preorder label is the root; split inorder around it to find the left and right subtrees. Apply the same idea to each part.

**Worked checkpoint.** Reconstruct the tree with preorder A, B, D, E, C, F and inorder D, B, E, A, C, F.

**Answer.** A is first in preorder, so it is root. Inorder splits into [D, B, E], A, [C, F]. The left subtree therefore has three nodes; its preorder is [B, D, E]. Its root is B, with left child D and right child E. The right subtree's preorder is [C, F]; its root is C, with only a right child F.

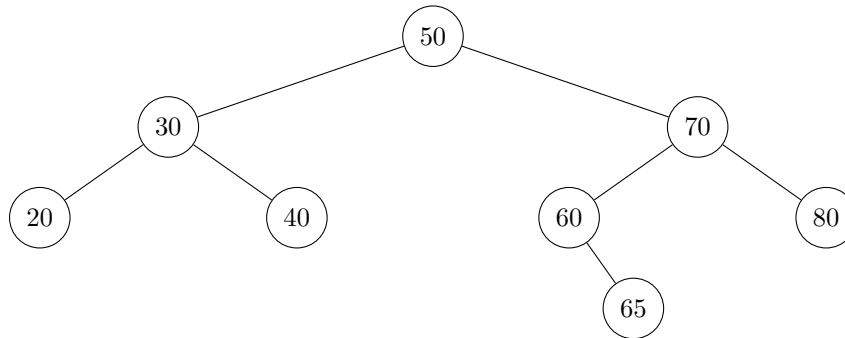


Inorder plus postorder also works: the *last* postorder label is the root. Preorder plus postorder alone does not generally determine a unique binary tree: A with only a left child B and A with only a right child B both give preorder A, B and postorder B, A.

**Exam habit:** After reconstructing a tree, regenerate both supplied traversals. This checks the entire structure, not just its root.

## 20 Binary search trees: the ordering rule and search

A **binary search tree (BST)** is a binary tree with an extra rule at *every node*: all keys in its left subtree are smaller, and all keys in its right subtree are larger. Both subtrees must themselves obey the rule. This guide rejects duplicate keys.



At root 50, every key on the left is below 50 and every key on the right is above 50. At node 70, keys 60 and 65 are in its left subtree and are below 70, even though 65 is the *right* child of 60.

### Algorithm: Search for key $k$ in a BST

**Step 1.** Start.

**Step 2.** Set  $P \leftarrow \text{root}$ .

**Step 3.** While  $P \neq \text{NULL}$ : if  $k = P.\text{key}$ , report found and go to Step 5; if  $k < P.\text{key}$ , set  $P \leftarrow P.\text{left}$ ; otherwise set  $P \leftarrow P.\text{right}$ .

**Step 4.** Report “not found.”

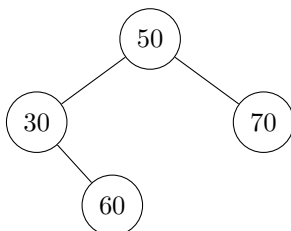
**Step 5.** End.

---

**Worked checkpoint.** Trace searches for 65 and 55 in the diagram.

**Answer.** For 65: compare with 50 (go right), 70 (go left), 60 (go right), then find 65: four key comparisons. For 55: 50  $\rightarrow$  70  $\rightarrow$  60  $\rightarrow$  left NULL, so it is absent. There are three comparisons with actual nodes.

### Why the rule applies to the whole subtree



Not a BST: 60 is greater than its parent 30, but it is in 50's left subtree and must be less than 50.

### Costs depend on shape

Search follows one path:  $O(h + 1)$  time and  $O(1)$  auxiliary space iteratively. This is commonly written  $O(h)$  for nontrivial trees. A balanced BST has height  $O(\log n)$ ; a skewed BST has height  $n - 1$ , so search is  $O(n)$ . An ordinary BST does not guarantee balance.

## 21 BST insertion, minimum, maximum and successor

**Algorithm: Insert key  $k$  into a BST (reject duplicates)**

**Step 1.** Start.

**Step 2.** Set  $P \leftarrow \text{root}$  and  $\text{parent} \leftarrow \text{NULL}$ .

**Step 3.** While  $P \neq \text{NULL}$ : if  $k = P.\text{key}$ , report duplicate and go to Step 8; otherwise set  $\text{parent} \leftarrow P$ , then move  $P$  left if  $k < P.\text{key}$  or right if  $k > P.\text{key}$ .

**Step 4.** Allocate node  $N$ . If allocation fails, report failure and go to Step 8.

**Step 5.** Set  $N.\text{key} \leftarrow k$  and both child pointers to  $\text{NULL}$ .

**Step 6.** If  $\text{parent} = \text{NULL}$ , set  $\text{root} \leftarrow N$ .

**Step 7.** Otherwise set  $\text{parent}.\text{left} \leftarrow N$  if  $k < \text{parent}.\text{key}$ ; else set  $\text{parent}.\text{right} \leftarrow N$ .

**Step 8.** End.

New keys are attached at an empty child position reached by comparisons, not at an arbitrary convenient spot. Existing nodes are not rearranged by ordinary BST insertion.

---

**Worked checkpoint.** Build a BST by inserting 50, 30, 70, 20, 40, 60, 80, 65.

**Answer.** 50 becomes root. 30 and 70 become its children. 20 and 40 become the children of 30. 60 and 80 become the children of 70. For 65, follow  $50 \rightarrow 70 \rightarrow 60$ ; 65 becomes 60's right child. The result is the preceding BST diagram.

**Algorithm: Find the minimum key in a BST**

**Step 1.** Start.

**Step 2.** If  $\text{root} = \text{NULL}$ , report an empty tree and go to Step 5.

**Step 3.** Set  $P \leftarrow \text{root}$ . While  $P.\text{left} \neq \text{NULL}$ , set  $P \leftarrow P.\text{left}$ .

**Step 4.** Output  $P.\text{key}$ .

**Step 5.** End.

For maximum, follow right pointers instead of left pointers. Minimum and maximum take  $O(h + 1)$  time, with  $O(1)$  extra space when written iteratively.

### Inorder successor and predecessor

The **inorder successor** is the next larger key. If a node has a right subtree, its successor is that subtree's minimum. Otherwise move up conceptually to the first ancestor for which the node lies in the left subtree; it may not exist. Parent pointers are optional: a search from root can remember a candidate ancestor.

The **inorder predecessor** is the next smaller key. With a left subtree, it is that subtree's maximum; otherwise it is the first qualifying ancestor reached from the right side.

---

**Worked checkpoint.** In the main BST, find  $\text{successor}(50)$ ,  $\text{successor}(65)$ ,  $\text{predecessor}(50)$ , and  $\text{successor}(80)$ .

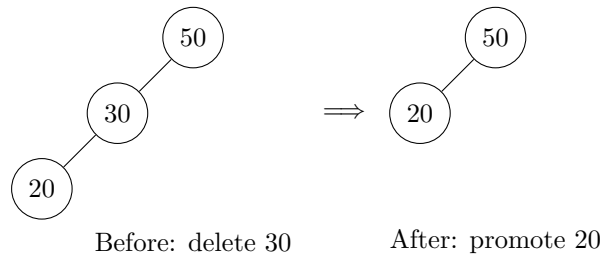
**Answer.** They are 60, 70, 40, and "none." For 65 there is no right subtree; 70 is the next larger ancestor. For 80, no larger key exists. Successor does not mean the nearest numerical value outside the tree.

**Exam habit:** Inserting sorted keys 10, 20, 30, 40 creates a right-skewed chain. Its height is 3, so ordinary BST operations are not automatically logarithmic.

## 22 BST deletion: understand the three cases

First find the node. The replacement must preserve the ordering of every remaining key.

1. **No children (leaf):** detach the node and replace its link with NULL.
2. **One child:** connect the parent directly to the node's only child. If deleting root, that child becomes root.
3. **Two children:** copy in the inorder successor's key (smallest key in the right subtree), then delete that successor node from the right subtree. The successor has no left child, so its removal reduces to case 1 or 2. The predecessor is an equally valid alternative when used consistently.



**Algorithm:**  $\text{Delete}(T, k)$ , returning the updated subtree root

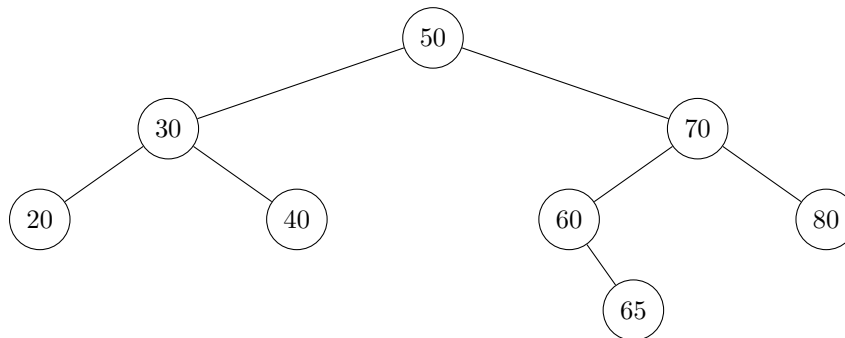
- Step 1.** Start.
- Step 2.** If  $T = \text{NULL}$ , set result to NULL (key absent) and go to Step 10.
- Step 3.** If  $k < T.\text{key}$ , set  $T.\text{left} \leftarrow \text{Delete}(T.\text{left}, k)$ , set result to  $T$ , and go to Step 10.
- Step 4.** If  $k > T.\text{key}$ , set  $T.\text{right} \leftarrow \text{Delete}(T.\text{right}, k)$ , set result to  $T$ , and go to Step 10.
- Step 5.** The key now matches. If  $T.\text{left} = \text{NULL}$ , save  $R \leftarrow T.\text{right}$ , free  $T$ , set result to  $R$ , and go to Step 10. This includes the leaf case.
- Step 6.** If  $T.\text{right} = \text{NULL}$ , save  $L \leftarrow T.\text{left}$ , free  $T$ , set result to  $L$ , and go to Step 10.
- Step 7.** Both children exist. Set  $S \leftarrow T.\text{right}$ ; while  $S.\text{left} \neq \text{NULL}$ , set  $S \leftarrow S.\text{left}$ .
- Step 8.** Copy  $S.\text{key}$  into  $T.\text{key}$ , then set  $T.\text{right} \leftarrow \text{Delete}(T.\text{right}, S.\text{key})$ .
- Step 9.** Set result to  $T$ .
- Step 10.** End and return result.

The top-level call must be `root = Delete(root, k)`. Returning a subtree root allows deletion to change a child link or the main root. If nodes contain additional records, copy the associated data consistently with the replacement key.

**Exam habit:** Do not delete an entire subtree when removing its parent. Preserve the child link in the one-child case. In the two-child case, copying a key without removing the original successor leaves a duplicate.

## 23 BST deletion: illustrated worked examples

Each example below starts independently from this same original BST:



### Case 1: delete the leaf 20

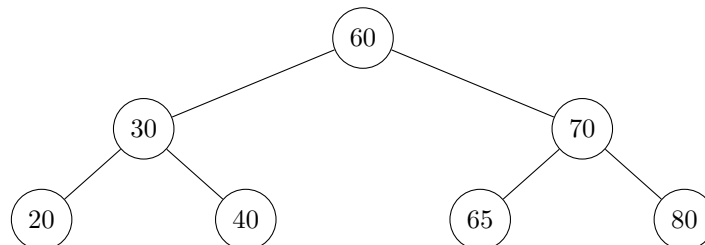
Follow  $50 \rightarrow 30 \rightarrow 20$ . Set  $30.\text{left}$  to NULL and free 20. No other link changes. The sorted order becomes 30, 40, 50, 60, 65, 70, 80.

### Case 2: delete 60, which has one child

Follow  $50 \rightarrow 70 \rightarrow 60$ . Set  $70.\text{left}$  to 65, then free 60. The entire remaining subtree is preserved. The sorted order becomes 20, 30, 40, 50, 65, 70, 80.

### Case 3: delete 50, which has two children

The successor is 60: go right to 70 and then left to 60. Copy 60 into the root. Remove the original 60 from under 70, promoting its right child 65. The result is:



Inorder is 20, 30, 40, 60, 65, 70, 80. Exactly one key disappeared, and the sequence remains strictly increasing. These two checks are useful for catching a lost subtree or duplicate successor.

---

**Worked checkpoint.** Why can the successor have a right child but not a left child?

**Answer.** It is the leftmost node in the right subtree. If it had a left child, a still smaller key in that subtree would exist, contradicting the choice of successor. Its right child is permitted and must be reattached when the successor is deleted.

### Time and space

Deletion follows a search path and possibly a successor path; total time is  $O(h + 1)$ . The recursive implementation uses  $O(h + 1)$  auxiliary stack space. Iterative deletion with parent tracking can use  $O(1)$  auxiliary space. Balanced-tree time is  $O(\log n)$ ; worst-case skewed-tree time is  $O(n)$ .

## 24 C example: BST search, deletion and cleanup

This function group uses the earlier TNode definition and `<stdlib.h>`. The algorithms remain the main exam reference; the code shows how returned child pointers preserve the tree.

```
TNode *searchBST(TNode *root, int key) {
    while (root != NULL && root->key != key) {
        if (key < root->key) root = root->left;
        else root = root->right;
    }
    return root; /* NULL means absent. */
}

TNode *deleteBST(TNode *root, int key) {
    if (root == NULL) return NULL;
    if (key < root->key) {
        root->left = deleteBST(root->left, key);
    } else if (key > root->key) {
        root->right = deleteBST(root->right, key);
    } else {
        if (root->left == NULL) {
            TNode *child = root->right;
            free(root);
            return child;
        }
        if (root->right == NULL) {
            TNode *child = root->left;
            free(root);
            return child;
        }
        TNode *successor = root->right;
        while (successor->left != NULL)
            successor = successor->left;
        root->key = successor->key;
        root->right = deleteBST(root->right, successor->key);
    }
    return root;
}

void destroyTree(TNode *root) {
    if (root == NULL) return;
    destroyTree(root->left);
    destroyTree(root->right);
    free(root);
}

/* Calls inside main:
   root = deleteBST(root, 50);
   destroyTree(root);
   root = NULL;
*/
```

**Worked checkpoint.** Why is `root->left = deleteBST(root->left, key)` necessary?

**Answer.** The left subtree's root may be removed or replaced by its child. The returned pointer records its new root; assigning it reconnects the parent. Ignoring it can leave the parent pointing to freed memory.

## 25 Fast revision: costs and common traps

### Complexity table

Here  $n$  is current size,  $h$  is tree height,  $w$  is maximum tree width, and  $C$  is array capacity. List operations

|                                                | Operation                                         | W      |
|------------------------------------------------|---------------------------------------------------|--------|
|                                                | Array / circular queue enqueue, dequeue, peek     | $O(1)$ |
|                                                | Linked queue enqueue/dequeue with both ends       | $O(1)$ |
|                                                | List search / traversal                           | $O(n)$ |
|                                                | Singly list insert/delete at beginning            | $O(1)$ |
|                                                | Singly list append without tail                   | $O(n)$ |
|                                                | Singly list append with tail                      | $O(1)$ |
| use only head unless a tail pointer is stated. | Singly list delete last (no predecessor supplied) | $O(n)$ |
|                                                | Doubly list delete known node                     | $O(1)$ |
|                                                | Doubly list delete by searched value              | $O(n)$ |
|                                                | Iterative linked-list reversal                    | $O(n)$ |
|                                                | Any complete traversal of a binary tree           | $O(n)$ |
|                                                | Recursive preorder/inorder/postorder              | $O(n)$ |
|                                                | Level-order traversal                             | $O(n)$ |
|                                                | BST iterative search / insert / min / max         | $O(h)$ |
|                                                | BST recursive deletion                            | $O(h)$ |

For insertion, the new node becomes part of the stored structure; the table counts only bounded temporary working storage. Total list/tree storage is  $O(n)$ ; a fixed array queue reserves  $O(C)$  even when empty.

### Questions an examiner can turn into traps

1. **“Queue full” without a representation.** State the chosen indices and full condition. Linear, circular-count and reserved-cell implementations differ.
2. **“Linked lists are always faster than arrays.”** False. Lists trade direct indexing for flexible links; the operation matters.
3. **“A doubly linked list makes searching faster.”** Not asymptotically. An unsorted search remains  $O(n)$ .
4. **“A full tree must be complete.”** False. Full concerns child count; complete concerns occupied positions.
5. **“Every binary tree has sorted inorder.”** False. The BST ordering rule is required.
6. **“BST search is always  $O(\log n)$ .”** False for an ordinary skewed BST.
7. **“To delete a two-child BST node, replace it with its right child.”** Unsafe: the right child’s left subtree may be lost or misplaced. Use a justified deletion method.
8. **“Depth equals height.”** Not generally. Depth looks upward to the root; height looks downward to the deepest leaf.

**Exam habit:** After pointer edits, trace both directions where applicable. After a BST edit, check that inorder is strictly increasing and that the number of nodes changes by exactly one when an insertion/deletion succeeds.

## 26 Practice paper: queues and linked lists (30 marks)

This is a 60-mark practice paper, not a prediction of your exam. Suggested time: 90 minutes for both practice pages. Draw diagrams and write numbered algorithms where requested. Detailed answers follow the question pages.

### Q1. Queue principles and linear overflow [5 marks]

- (a) Define FIFO, enqueue, dequeue, overflow and underflow. [2]
- (b) A linear array queue of capacity 4 starts empty. Enqueue 5, 10, 15, 20; then dequeue twice. Give front, rear and the live contents using this guide's linear convention. Can you enqueue 25? Explain and name a better representation for reusing the free cells. [3]

### Q2. Circular queue trace and algorithm [7 marks]

Use capacity 4 and the count-based convention: front = rear = count = 0 initially. Perform:

enqueue(10), enqueue(20), enqueue(30), dequeue(),  
enqueue(40), enqueue(50), enqueue(60), dequeue(), enqueue(60).

- (a) Give front, rear, count and logical queue order after *every* operation. Identify any failure. [4]
- (b) Write the numbered circular-enqueue algorithm, including Start, full check, wraparound and End. [3]

### Q3. Singly linked-list edits [6 marks]

Begin with  $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$ .

- (a) Insert 15 at position 2; delete the first occurrence of 30; then delete the first node. Draw the list after each operation. [3]
- (b) Write the step-wise algorithm to delete the first node, including the empty case and memory release. State the time cost. [3]

### Q4. Reverse a singly linked list [6 marks]

Reverse  $2 \rightarrow 4 \rightarrow 6 \rightarrow \text{NULL}$  without allocating nodes or swapping data.

- (a) Write a numbered algorithm. [3]
- (b) Show previous and current after each loop iteration. Explain the saved-next pointer, and give time and auxiliary-space costs. [3]

### Q5. Doubly linked-list deletion [6 marks]

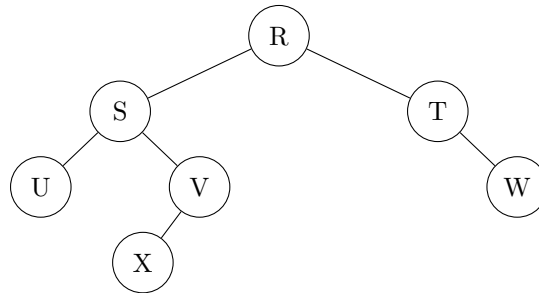
- (a) Given  $5 \leftrightarrow 10 \leftrightarrow 15$ , with head at 5 and tail at 15, delete the known node 10. Draw and describe both changed links. [2]
- (b) Write a general numbered algorithm for deleting a known node that also handles head, tail and the only node. [3]
- (c) Explain why deletion by a supplied value can still take  $O(n)$ . [1]

## 27 Practice paper: trees and BSTs (30 marks)

### Q6. Terminology and formulas

[6 marks]

Use this tree, with depth and height measured in edges:



- Name the root, V's parent, S's sibling, leaves, and ancestors of X. [2]
- Find  $\text{depth}(V)$ ,  $\text{height}(S)$ , tree height and number of edges. [2]
- Give the maximum number of nodes in a binary tree of this height and the number of leaves in a perfect binary tree of this height. [2]

### Q7. Traversals and their working storage

[6 marks]

For Q6's tree, give preorder, inorder, postorder and level order. [4] Explain which traversal uses a queue and state worst-case time and auxiliary space for level order. [2]

### Q8. Build and search a BST

[5 marks]

Insert 40, 20, 60, 10, 30, 50, 70, 55 into an empty BST.

- Draw the resulting BST and give its inorder traversal. [3]
- Trace search for 55, then search for 25. [2]

### Q9. Delete from a BST

[7 marks]

Starting from Q8's original BST, delete 10, then 50, then 40, in that order. Use the inorder successor for a node with two children.

- Identify the deletion case each time and draw the tree after each deletion. [5]
- Explain why the final tree remains a BST, and give worst-case deletion time in terms of  $h$  and  $n$ . [2]

### Q10. Explain or correct these claims

[6 marks]

Give one clear reason or counterexample for each (1 mark each).

- Every full binary tree is complete.
- An ordinary BST always gives logarithmic search time.
- Checking only immediate children is enough to validate a BST.
- A singly linked list with a tail pointer always permits  $O(1)$  tail deletion.
- Preorder and postorder uniquely determine every binary tree.
- After `free(p)`, it is safe to read `p->next` if the program has not allocated anything else.

## 28 Detailed answers: Q1 and Q2

### Q1. Queue principles and linear overflow

(a) FIFO means the earliest inserted item is removed first. Enqueue adds at the rear; dequeue removes at the front. Overflow means an insertion cannot be accepted because the implementation is full or cannot allocate storage. Underflow means removal is attempted when empty.

(b) After four insertions,  $\text{front} = 0$  and  $\text{rear} = 3$ . Removing 5 advances front to 1; removing 10 advances it to 2. Rear remains 3. The live queue is 15, 20.

|        |        | front | rear |
|--------|--------|-------|------|
| unused | unused | 15    | 20   |
| 0      | 1      | 2     | 3    |

Enqueue(25) fails because  $\text{rear} = C - 1 = 3$ . A circular queue reuses indices 0 and 1. A full answer distinguishes the two unused physical cells from the implementation's blocked insertion index.

### Q2. Circular queue trace

| Operation            | front | rear | count | Logical order  |
|----------------------|-------|------|-------|----------------|
| Initial              | 0     | 0    | 0     | empty          |
| Enqueue 10           | 0     | 1    | 1     | 10             |
| Enqueue 20           | 0     | 2    | 2     | 10, 20         |
| Enqueue 30           | 0     | 3    | 3     | 10, 20, 30     |
| Dequeue: returns 10  | 1     | 3    | 2     | 20, 30         |
| Enqueue 40           | 1     | 0    | 3     | 20, 30, 40     |
| Enqueue 50           | 1     | 1    | 4     | 20, 30, 40, 50 |
| Enqueue 60: fails    | 1     | 1    | 4     | unchanged      |
| Dequeue: returns 20  | 2     | 1    | 3     | 30, 40, 50     |
| Enqueue 60: succeeds | 2     | 2    | 4     | 30, 40, 50, 60 |

The failed insertion changes nothing. The last physical array is [50, 60, 30, 40], but FIFO order starts at  $\text{front} = 2$ : 30, 40, 50, 60. Confusing array order with logical order loses the key point of a circular queue.

### Algorithm: Model answer: circular enqueue( $x$ )

**Step 1.** Start.

**Step 2.** If  $\text{count} = C$ , report overflow and go to Step 6.

**Step 3.** Set  $Q[\text{rear}] \leftarrow x$ .

**Step 4.** Set  $\text{rear} \leftarrow (\text{rear} + 1) \bmod C$ .

**Step 5.** Set  $\text{count} \leftarrow \text{count} + 1$ .

**Step 6.** End.

Marking focus: a consistent rear convention, full check before writing, modulo wraparound, and unchanged state on failure.

## 29 Detailed answers: Q3 and Q4

### Q3. Singly linked-list edits

(a) The three successive states are:

| Operation                     | Result                                                                    |
|-------------------------------|---------------------------------------------------------------------------|
| Insert 15 at position 2       | $10 \rightarrow 15 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$ |
| Delete first occurrence of 30 | $10 \rightarrow 15 \rightarrow 20 \rightarrow \text{NULL}$                |
| Delete the first node         | $15 \rightarrow 20 \rightarrow \text{NULL}$                               |

For insertion, `new15.next` is first set to `node20`, then `node10.next` is set to `new15`. To remove 30, `node20.next` is set to `NULL` before freeing 30. The final deletion changes head to `node15`.

#### Algorithm: Model answer: delete first node

**Step 1.** Start.

**Step 2.** If `head = NULL`, report empty and go to Step 6.

**Step 3.** Set  $T \leftarrow \text{head}$  and save  $x \leftarrow T.\text{data}$ .

**Step 4.** Set  $\text{head} \leftarrow T.\text{next}$ .

**Step 5.** Free  $T$  and output  $x$ .

**Step 6.** End.

Time is  $O(1)$  and auxiliary space is  $O(1)$ . If there was only one node, the new head is `NULL`, which correctly represents an empty list.

### Q4. Reverse without allocating nodes

#### Algorithm: Model answer: reverse a singly linked list

**Step 1.** Start.

**Step 2.** Set  $\text{previous} \leftarrow \text{NULL}$  and  $\text{current} \leftarrow \text{head}$ .

**Step 3.** While  $\text{current} \neq \text{NULL}$ : save  $\text{savedNext} \leftarrow \text{current.next}$ ; set  $\text{current.next} \leftarrow \text{previous}$ ; set  $\text{previous} \leftarrow \text{current}$ ; set  $\text{current} \leftarrow \text{savedNext}$ .

**Step 4.** Set  $\text{head} \leftarrow \text{previous}$ .

**Step 5.** End.

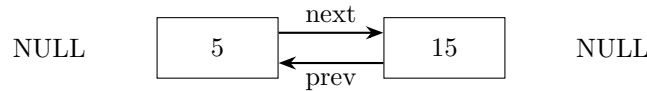
| After iteration | previous points to                                      | current points to                         |
|-----------------|---------------------------------------------------------|-------------------------------------------|
| 1               | $2 \rightarrow \text{NULL}$                             | $4 \rightarrow 6 \rightarrow \text{NULL}$ |
| 2               | $4 \rightarrow 2 \rightarrow \text{NULL}$               | $6 \rightarrow \text{NULL}$               |
| 3               | $6 \rightarrow 4 \rightarrow 2 \rightarrow \text{NULL}$ | $\text{NULL}$                             |

The first iteration changes `2.next` from 4 to `NULL`. Without saving the original pointer to 4, the remainder of the list would be lost. Each iteration processes one node, giving  $O(n)$  time; only a fixed number of pointers is needed, giving  $O(1)$  auxiliary space. The data 2, 4 and 6 never changes.

## 30 Detailed answers: Q5 and Q6

### Q5. Doubly linked-list deletion

Set 5.next to 15 and 15.prev to 5, then free 10. Head remains 5, tail remains 15, head.prev remains NULL, and tail.next remains NULL.



**Algorithm: Model answer: delete a known valid node  $P$**

**Step 1.** Start.

**Step 2.** If  $P = \text{NULL}$ , report failure and go to Step 6.

**Step 3.** If  $P.\text{prev} \neq \text{NULL}$ , set  $P.\text{prev}.\text{next} \leftarrow P.\text{next}$ ; otherwise set  $\text{head} \leftarrow P.\text{next}$ .

**Step 4.** If  $P.\text{next} \neq \text{NULL}$ , set  $P.\text{next}.\text{prev} \leftarrow P.\text{prev}$ ; otherwise set  $\text{tail} \leftarrow P.\text{prev}$ .

**Step 5.** Free  $P$ .

**Step 6.** End.

When  $P$  is the only node, both head and tail become NULL. When  $P$  is at an end, one boundary pointer changes; in the middle, both neighbours change. A supplied *value* must first be located; a worst-case traversal examines all  $n$  nodes, so total deletion-by-value time is  $O(n)$  even though relinking takes  $O(1)$ .

### Q6. Terminology and formulas

- Root: R. V's parent: S. S's sibling: T.
- Leaves: U, X, W, because each has no children.
- X's ancestors: V, S, R, following the path upward.
- $\text{Depth}(V) = 2$  from the two edges R–S–V.
- $\text{Height}(S) = 2$  from the longest downward path S–V–X.
- Tree height = 3 from R–S–V–X.
- There are 7 nodes, so there are  $7 - 1 = 6$  edges.

At height 3, the maximum possible number of binary-tree nodes is

$$1 + 2 + 4 + 8 = 2^{3+1} - 1 = 15.$$

A perfect binary tree of this height has  $2^3 = 8$  leaves. These are properties of a *perfect tree of the same height*, not counts for the supplied seven-node tree.

The supplied tree is not full: V and T each have exactly one child. It is not complete: T has a right child with no left child, leaving a gap before W. This extra check is a useful way to separate shape definitions from node-count formulas.

## 31 Detailed answers: Q7 and Q8

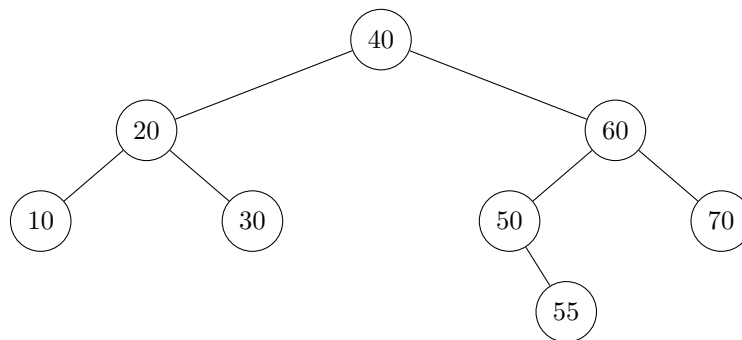
### Q7. Traversals

| Traversal   | Output              |
|-------------|---------------------|
| Preorder    | R, S, U, V, X, T, W |
| Inorder     | U, S, X, V, R, T, W |
| Postorder   | U, X, V, S, W, T, R |
| Level order | R, S, T, U, V, W, X |

In preorder, R is visited first, followed by the complete S subtree. In inorder, V's left child X comes before V. In postorder, X precedes V, and both precede S. Level order finishes U, V and W at depth 2 before visiting X at depth 3.

Level order uses a **queue**. After removing R, the queue is [S, T]. After S it is [T, U, V]; after T it is [U, V, W]; after U it is [V, W]; after V it is [W, X]; after W it is [X]; after X it is empty. Every node is enqueued and dequeued once, so time is  $O(n)$ . Queue space is  $O(w)$ , worst-case  $O(n)$ .

### Q8. Construct the BST



The last inserted value 55 follows right from 40, left from 60, and right from 50. Inorder is

10, 20, 30, 40, 50, 55, 60, 70.

The strictly increasing output checks the BST rule for this tree.

**Search for 55:** compare 55 with 40 (larger: right), 60 (smaller: left), 50 (larger: right), then 55 (equal: found). Four nodes are examined.

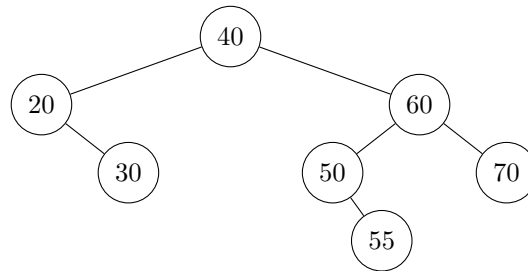
**Search for 25:** compare 25 with 40 (left), 20 (right), 30 (left), then reach NULL. It is absent. Three actual keys are compared. The empty child is the position where a future insertion of 25 would attach.

**Exam habit:** A traversal lists every node; a BST search lists only the nodes on one decision path. Do not include nodes from the discarded subtrees in a search trace.

## 32 Detailed answers: Q9, three successive deletions

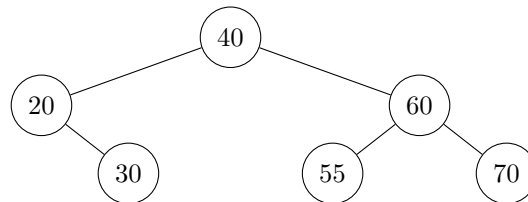
### First remove 10: leaf case

Set 20.left to NULL and free 10. All other links remain the same.



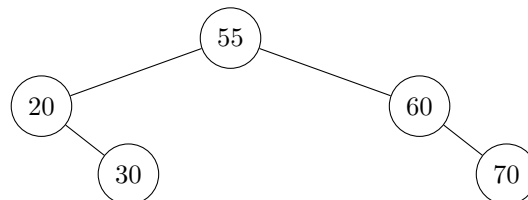
### Then remove 50: one-child case

50 has the single right child 55. Set 60.left to 55, then free 50.



### Finally remove 40: two-child case

Use the *current* tree: the successor is now 55, not 50. Copy 55 into the root, then remove the original leaf 55 by setting 60.left to NULL.



Final inorder is 20, 30, 55, 60, 70. All left-subtree keys remain smaller than their root, all right-subtree keys remain larger, and each subtree is still a BST. The original eight nodes have become five after three successful deletions.

Each deletion takes  $O(h + 1)$  time (often abbreviated  $O(h)$ ). In a balanced tree this is  $O(\log n)$ ; in the worst-case shape of an ordinary BST it is  $O(n)$ . Do not use a balanced-tree bound without a balancing assumption.

### 33 Detailed answers: Q10 and final self-check

#### Q10. Every claim is false

- (a) A full tree can be unevenly filled. A root with a left leaf and a right child that has two children is full, but its last level has nodes on the right with empty positions on the left, so it is not complete.
- (b) Inserting sorted keys 10, 20, 30, 40 into an ordinary BST creates a chain. Searching for 40 examines all four nodes. For  $n$  sorted insertions the height is  $n - 1$ , and worst-case search takes  $O(n)$ .
- (c) A tree with root 50, left child 30, and 30's right child 60 passes immediate-child comparisons but fails the whole-subtree rule: 60 is in 50's left subtree. Validating a BST requires inherited lower and upper limits, or an equivalent global check.
- (d) A tail pointer locates the last singly linked node, but it does not locate its predecessor. Updating the predecessor's next link generally requires an  $O(n)$  traversal.
- (e) Root A with only left child B and root A with only right child B both yield preorder A, B and postorder B, A. The traversals alone cannot tell which child position is occupied.
- (f) Reading `p->next` after freeing `p` is use-after-free and has undefined behaviour in C. A value appearing unchanged does not make the access legal. Save the next pointer before freeing.

#### Can you do all of this without looking?

- ☐ Define FIFO and trace queue insertion/removal, including overflow and underflow.
- ☐ State your front/rear convention and explain circular wraparound.
- ☐ Draw a node and distinguish data, pointer, address and NULL.
- ☐ Insert and delete at both ends and at a position in a singly linked list.
- ☐ Reverse a singly linked list using three pointers and explain the assignment order.
- ☐ Update both directions in a doubly linked list and handle a one-node deletion.
- ☐ Define root, parent, child, sibling, leaf, subtree, degree, depth and height.
- ☐ Distinguish binary, full, complete, perfect and skewed trees with examples.
- ☐ Use  $2^d$ ,  $2^{h+1} - 1$ ,  $n - 1$  edges, and the full-tree leaf relation correctly.
- ☐ Produce all four traversals and explain why level order needs a queue.
- ☐ Build a BST, trace successful and unsuccessful searches, and find min/max.
- ☐ Delete BST nodes with zero, one and two children without losing subtrees.
- ☐ Explain each operation's time cost, including search and recursive stack use.

#### Your final answer-writing routine

**Define** → **state assumptions** → **draw** → **Start** → **numbered steps** → **End** → **dry run** → **complexity**. Check empty, one-item and normal cases. For queues also check full and wraparound; for lists check first and last nodes; for BSTs check deletion of the root and an absent key.